

Software SPI (Bit-Banging) Driver Design

STM32F103xB - Based directly on `hal_soft_spi.h` / `hal_soft_spi.c`

Mục tiêu: đọc xong hiểu bản chất driver, biết mỗi API làm gì, CPOL/CPHA chạy ra sao, cách dùng CS, cách test, và các giới hạn kỹ thuật của code hiện tại.

Mental Model - Understand the Driver in 30 Seconds

- 1 Init**
Turn GPIO pins into SCK/MOSI/MISO/CS and set idle levels.
- 2 CS LOW**
Select exactly one SPI slave.
- 3 For each bit**
Set MOSI -> toggle SCK -> sample MISO -> toggle SCK back.
- 4 Repeat 8 times**
One transmitted byte and one received byte are completed.
- 5 Buffer APIs**
Repeat the byte engine for N bytes.
- 6 CS HIGH**
Close the transaction.

Software SPI = CPU manually creates the same four wires that a hardware SPI peripheral would generate automatically.

1. Bản chất của Software SPI

Software SPI (bit-banging) không dùng peripheral SPI phần cứng. CPU tự tạo SCK, MOSI, CS bằng cách ghi GPIO và tự đọc MISO bằng GPIO. Vì vậy mọi cạnh clock trong giao tiếp đều do code tạo ra.

Software SPI (Bit-Banging) - Architecture

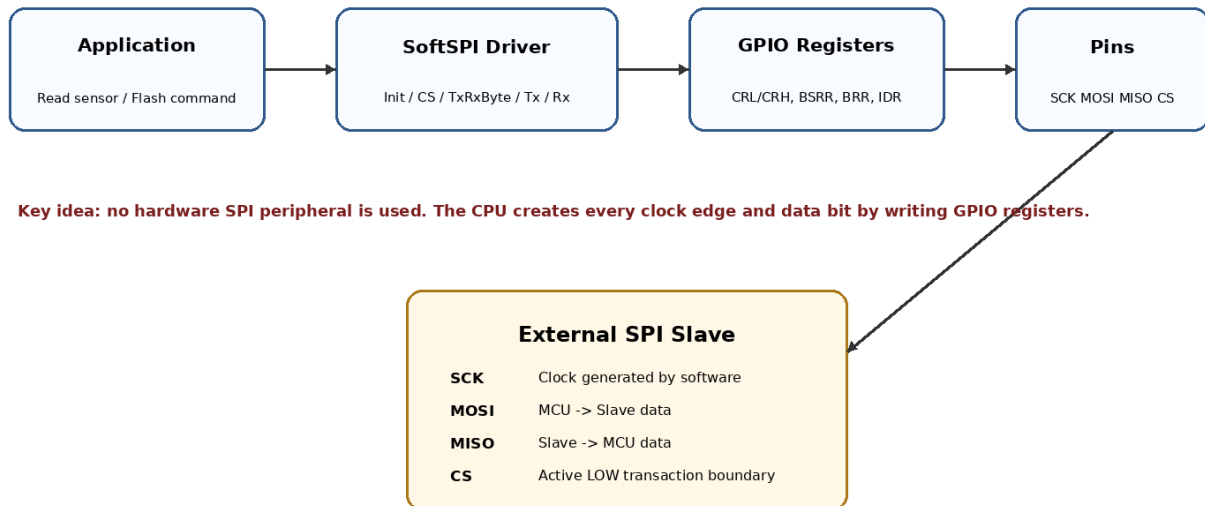


Figure 1. Kiến trúc thực tế của code SoftSPI.

Điểm cốt lõi: hardware SPI làm việc này bằng state machine trong silicon; SoftSPI làm cùng logic bằng vòng lặp C + GPIO register.

Ưu điểm: dùng được gần như bất kỳ GPIO nào, rất linh hoạt khi debug/prototype. **Nhược điểm:** tốn CPU, tốc độ phụ thuộc code/compiler/clock, timing kém chính xác hơn hardware SPI.

2. Cấu trúc dữ liệu trong hal_soft_spi.h

Type	Fields	Ý nghĩa
SoftSPI_PinConfigTypeDef	SckPort/SckPin, MosiPort/MosiPin, MisoPort/MisoPin, CsPort/CsPin	Cho phép ánh xạ 4 dây SPI tới GPIO tùy chọn.
SoftSPI_InitTypeDef	Pins, CPOL, CPHA, FirstBit, DelayUs	Toàn bộ cấu hình protocol và tốc độ software.
SoftSPI_HandleTypeDef	Init + State	Handle truyền xuống mọi API; State=1U sau Init.

Các macro CPOL/CPHA/FirstBit chỉ có hai giá trị 0/1. Driver hiện chỉ truyền frame 8-bit, không có data-size 16-bit.

3. GPIO layer - code đang điều khiển chân thế nào?

Driver truy cập GPIO trực tiếp thay vì gọi một HAL GPIO khác:

```
SoftSPI_WritePin(): PinState=1 -> GPIOx->BSRR = (1U << Pin) PinState=0 -> GPIOx->BRR = (1U << Pin)
SoftSPI_ReadPin(): đọc GPIOx->IDR
SoftSPI_ConfigPinMode(): chỉnh 4-bit cấu hình pin trong CRL/CRH
```

Trong Init, SCK/MOSI/CS được đặt Output Push-Pull 10 MHz (Mode=0x1), còn MISO là Input Floating (Mode=0x4). Code không bật RCC GPIO clock, vì vậy đây là precondition: clock của các GPIO port phải được enable trước khi gọi SoftSPI_Init().

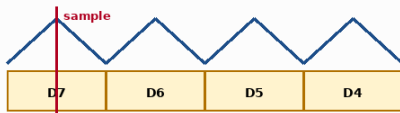
4. CPOL, CPHA và 4 SPI Mode

CPOL quyết định mức SCK khi idle. CPHA quyết định dữ liệu được sample ở cạnh thứ nhất hay cạnh thứ hai. Hai bit này tạo ra 4 SPI modes.

SPI Modes 0..3 - CPOL/CPHA in the Current Code

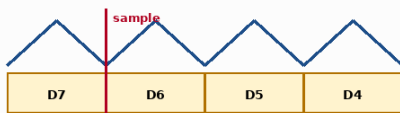
Mode 0: CPOL=0, CPHA=0

SCK idle=LOW | MOSI drive=before 1st edge | MISO sample=1st edge



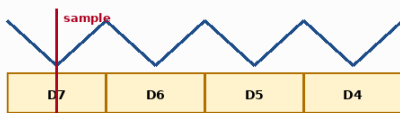
Mode 1: CPOL=0, CPHA=1

SCK idle=LOW | MOSI drive=1st edge | MISO sample=2nd edge



Mode 2: CPOL=1, CPHA=0

SCK idle=HIGH | MOSI drive=before 1st edge | MISO sample=1st edge



Mode 3: CPOL=1, CPHA=1

SCK idle=HIGH | MOSI drive=1st edge | MISO sample=2nd edge

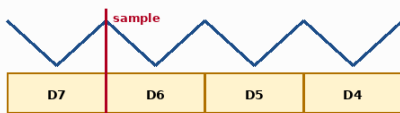


Figure 2. Cách code hiện tại tạo Mode 0/1/2/3.

Mode	CPOL	CPHA	Idle	Behavior trong code
Mode 0	0	0	SCK idle LOW	Drive data trước cạnh 1; sample cạnh 1
Mode 1	0	1	SCK idle LOW	Drive data ở cạnh 1; sample cạnh 2
Mode 2	1	0	SCK idle HIGH	Drive data trước cạnh 1; sample cạnh 1
Mode 3	1	1	SCK idle HIGH	Drive data ở cạnh 1; sample cạnh 2

Muốn giao tiếp đúng, mode của SoftSPI phải match mode mà SPI slave yêu cầu trong datasheet.

5. FirstBit - MSB first hay LSB first

Mỗi byte có 8 bit. Code tính bitPos theo FirstBit:

MSB first: bitPos = 7 - i -> D7, D6, ... D0 LSB first: bitPos = i -> D0, D1, ... D7

Ví dụ txData=0xA5 = binary 1010 0101. Với MSB first, chuỗi đi ra MOSI là 1 0 1 0 0 1 0 1. Với LSB first, thứ tự được đảo theo vị trí bit.

6. API lỗi - SoftSPI_TransmitReceiveByte()

Đây là engine quan trọng nhất. SPI là full-duplex: mỗi clock bit vừa đẩy 1 bit MOSI ra, vừa có thể nhận 1 bit MISO vào. Vì thế API byte-level vừa transmit vừa receive trong cùng một vòng 8 bit.

One Byte Transfer - What SoftSPI_TransmitReceiveByte() Really Does

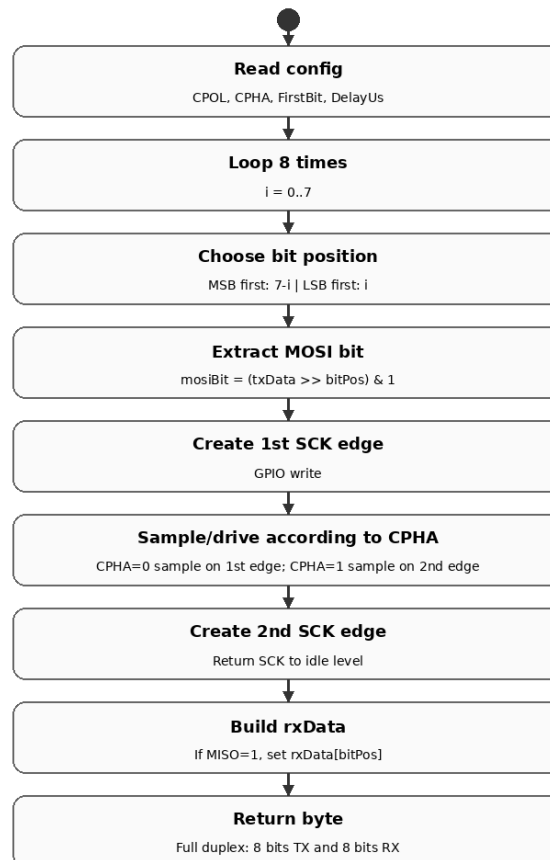


Figure 3. Flow một byte theo đúng code.

Với CPHA=0, MOSI được đặt trước cạnh clock thứ nhất và MISO được sample tại cạnh thứ nhất. Với CPHA=1, code tạo cạnh thứ nhất rồi mới đặt MOSI; MISO được sample tại cạnh thứ hai.

rxData chỉ set bit khi MISO đọc được mức 1; vì rxData khởi tạo 0 nên bit nhận mức 0 không cần thao tác.

7. DelayUs thực sự có phải microsecond không?

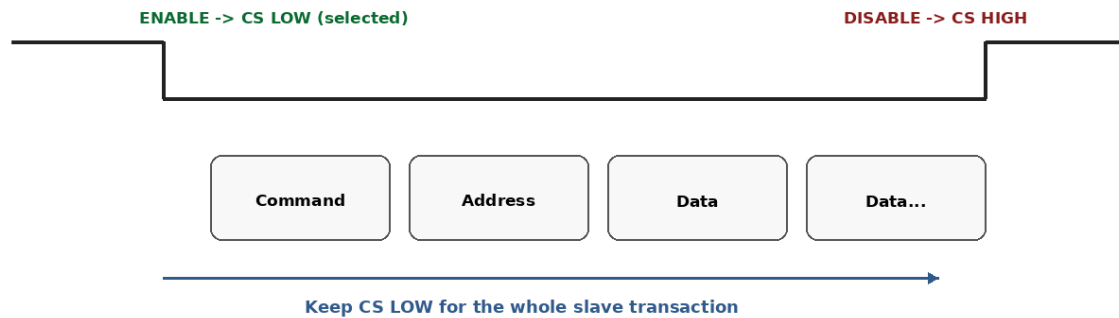
Trong handle, field tên là DelayUs, nhưng implementation chỉ gọi một vòng lặp NOP:

```
static void SoftSPI_Delay(uint32_t count) { while (count--) { asm("nop"); } }
```

Vì vậy giá trị này không phải microsecond tuyệt đối. Thời gian thực phụ thuộc CPU clock, compiler optimization, chi phí vòng lặp và memory/register access. Tên chính xác hơn là DelayCount, hoặc phải calibrate/tạo timer delay nếu muốn đơn vị us thật.

8. Chip Select - ai điều khiển transaction?

Chip Select (CS) - Transaction Boundary



Important: SoftSPI_Transmit() and SoftSPI_Receive() do NOT change CS automatically.

Caller must: ChipSelect(ENABLE) -> transfer API(s) -> ChipSelect(DISABLE).

Figure 4. CS active LOW trong thiết kế hiện tại.

SoftSPI_ChipSelect(hspi, ENABLE) kéo CS xuống LOW; DISABLE kéo CS HIGH. Đây là semantics hơi khác trực giác: ENABLE nghĩa là select slave, không phải đưa pin HIGH.

Quan trọng: SoftSPI_Transmit() và SoftSPI_Receive() không tự điều khiển CS. Caller phải giữ CS LOW xuyên suốt một command của slave.

```
SoftSPI_ChipSelect(&hspi, ENABLE); // CS LOW SoftSPI_Transmit(&hspi, cmd, cmd_len);  
SoftSPI_Receive(&hspi, rx, rx_len); SoftSPI_ChipSelect(&hspi, DISABLE); // CS HIGH
```

9. Thiết kế từng Public API

API	What it does	Precondition	Return
SoftSPI_Init	Validate hspi; config GPIO; CS HIGH; SCK idle by CPOL; MOSI LOW; State=1	GPIO clocks must already be enabled	STD_OK / STD_ERROR
SoftSPI_ChipSelect	Assert/deassert CS	hspi valid	STD_OK / STD_ERROR
SoftSPI_TransmitReceiveByte	Transfer exactly 8 bits full-duplex	hspi valid + initialized	rx byte
SoftSPI_Transmit	Loop byte engine; discard received bytes	hspi, pData valid; Size>0	STD_OK / STD_ERROR
SoftSPI_Receive	Loop byte engine with tx dummy 0xFF	hspi, pData valid; Size>0	STD_OK / STD_ERROR

API Relationship - Public APIs vs Private Helpers

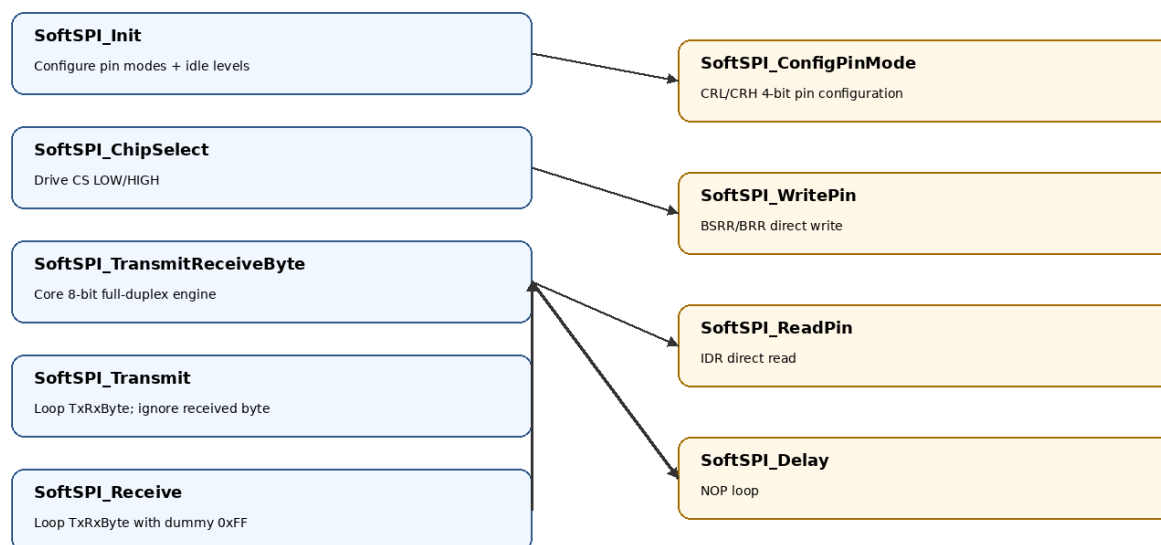


Figure 5. Quan hệ giữa public API và private helpers.

10. Workflows sử dụng thực tế

10.1 Ghi command/data vào slave

```
SoftSPI_ChipSelect(&hspi;, ENABLE); SoftSPI_Transmit(&hspi;, tx, tx_len);  
SoftSPI_ChipSelect(&hspi;, DISABLE);
```

10.2 Đọc register từ sensor

Example Sequence - Read One Register from an SPI Sensor

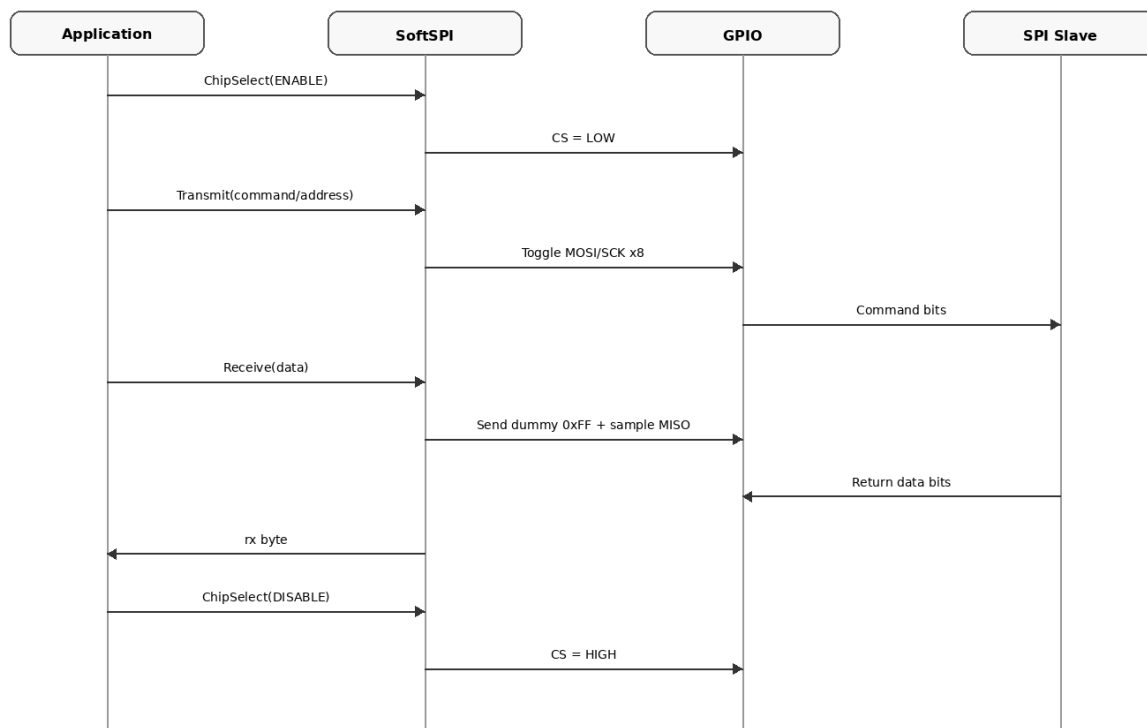


Figure 6. Ví dụ sequence đọc một register.

Điểm cần hiểu: để nhận dữ liệu trên SPI, master vẫn phải tạo clock. Vì vậy SoftSPI_Receive() gửi dummy 0xFF trên MOSI để tạo 8 clock cho mỗi byte nhận.

11. Vì sao Receive gửi dummy 0xFF?

SPI không có khái niệm "chỉ nhận mà không phát clock" đối với master. Mỗi bit MISO chỉ xuất hiện khi master tạo SCK. Vì thế driver gọi:

```
pData[i] = SoftSPI_TransmitReceiveByte(hspi, 0xFFU);
```

0xFF chỉ là dummy byte. Slave có thể bỏ qua MOSI trong pha read. Một số protocol có thể dùng dummy 0x00 hoặc giá trị khác; code hiện fix cứng 0xFF.

12. State và validation

SoftSPI_Init() set hspi->State = 1U để biểu diễn Ready, nhưng các API còn lại chưa kiểm tra State. Đây là một điểm thiết kế chưa hoàn chỉnh.

Các API buffer kiểm tra NULL/Size, nhưng SoftSPI_TransmitReceiveByte() không kiểm tra hspi=NULL. Vì nó public trong header, caller có thể gọi trực tiếp và gây fault.

13. Technical gaps / điểm nên cải thiện

ID	Gap	Risk	Recommended design
GAP-01	DelayUs không phải us thật	Timing phụ thuộc CPU/compiler	Đổi thành DelayCount hoặc dùng timer/cycle counter để calibrate.
GAP-02	Không enable GPIO clock	Init có thể ghi vào port chưa clock	Thêm RCC dependency hoặc document precondition rõ.

ID	Gap	Risk	Recommended design
GAP-03	Không validate Pin 0..15	Pin sai có thể shift/register config sai	Validate tất cả pin.
GAP-04	Không validate CPOL/CPHA/FirstBit	Giá trị ngoài 0/1 có behavior không rõ	Thêm validation macro.
GAP-05	TxRxByte không check hspi	Public API có thể dereference NULL	Thêm NULL/state validation hoặc make private.
GAP-06	State dùng magic 1U	Khó trace/read	Tạo SOFT_SPI_STATE_READY/BUSY/RESET enum.
GAP-07	Không lock/concurrency	IRQ/task khác có thể xen giữa edges	Critical section/mutex nếu dùng RTOS/ISR.
GAP-08	Tốc độ không được guarantee	Bit-bang jitter	Đo logic analyzer; dùng hardware SPI nếu timing chặt.
GAP-09	CS không tự động	Caller có thể quên/toggle sai transaction	Document API contract hoặc thêm TransactionBegin/End.
GAP-10	Chỉ 8-bit frame	Không hỗ trợ word 16-bit native	Thêm TxRx16 nếu use case cần.

14. Test plan để chứng minh driver chạy đúng

Test ID	Use case	Expected	How to observe
SSPI-TC-001	Init CPOL=0	CS HIGH, SCK LOW, MOSI LOW	Multimeter/logic analyzer
SSPI-TC-002	Init CPOL=1	CS HIGH, SCK HIGH	Logic analyzer
SSPI-TC-003	Mode 0	Sample on first edge, idle LOW	Logic analyzer vs known SPI slave
SSPI-TC-004	Mode 1	Sample on second edge, idle LOW	Logic analyzer
SSPI-TC-005	Mode 2	Sample on first edge, idle HIGH	Logic analyzer
SSPI-TC-006	Mode 3	Sample on second edge, idle HIGH	Logic analyzer
SSPI-TC-007	MSB first send 0xA5	MOSI sequence D7..D0	Logic analyzer
SSPI-TC-008	LSB first send 0xA5	MOSI sequence D0..D7	Logic analyzer
SSPI-TC-009	Loopback MOSI->MISO	Rx byte equals Tx byte	Wire MOSI to MISO
SSPI-TC-010	Receive 4 bytes	32 clocks generated, returned bytes correct	Known slave or pattern generator
SSPI-TC-011	CS transaction	CS stays LOW across command+data	Logic analyzer
SSPI-TC-012	NULL/Size=0	Buffer APIs return STD_ERROR	Unit test

15. Loopback test đơn giản nhất

Nối MOSI trực tiếp sang MISO. Khi transmit 0xA5, cùng các cạnh SCK đó driver sẽ đọc lại chính bit vừa phát. Nếu CPOL/CPHA/FirstBit logic đúng, rxData phải bằng txData.

```
uint8_t rx; SoftSPI_ChipSelect(&hspi, ENABLE); rx = SoftSPI_TransmitReceiveByte(&hspi, 0xA5U);
SoftSPI_ChipSelect(&hspi, DISABLE); /* Expected: rx == 0xA5 */
```

Loopback kiểm tra được bit order, edge generation và GPIO read/write, nhưng chưa chứng minh timing tương thích với một slave thực. Sau loopback nên test với chip thật + logic analyzer.

16. Requirement-style mapping từ code sang design

Requirement ID	Requirement	Code mapping
SSPI-FR-001	Driver shall initialize SCK/MOSI/CS as outputs and MISO as input.	SoftSPI_Init / SoftSPI_ConfigPinMode
SSPI-FR-002	Driver shall set SCK idle level according to CPOL.	SoftSPI_Init
SSPI-FR-003	Driver shall support CPHA first-edge and second-edge sampling.	SoftSPI_TransmitReceiveByte
SSPI-FR-004	Driver shall support MSB-first and LSB-first transfer.	SoftSPI_TransmitReceiveByte
SSPI-FR-005	Driver shall transfer one 8-bit frame full-duplex.	SoftSPI_TransmitReceiveByte
SSPI-FR-006	Driver shall transmit a buffer by repeating byte transfers.	SoftSPI_Transmit
SSPI-FR-007	Driver shall receive a buffer by generating clock with dummy bytes.	SoftSPI_Receive
SSPI-FR-008	Driver shall provide active-low chip-select control.	SoftSPI_ChipSelect
SSPI-NFR-001	Driver shall avoid dynamic memory.	All source
SSPI-NFR-002	GPIO access shall be direct and deterministic at register level.	WritePin/ReadPin/ConfigPinMode
SSPI-NFR-003	Transfer timing shall be measurable and documented for target CPU/compiler.	Delay mechanism - current gap

17. Khi nào nên dùng SoftSPI, khi nào dùng Hardware SPI?

Choice	Good fit	Trade-off
SoftSPI	Pin linh hoạt; prototype; debug; tốc độ thấp/trung bình; cần protocol nhỏ	Tốn CPU; timing/jitter phụ thuộc software
Hardware SPI	Flash/display tốc độ cao; DMA; nhiều dữ liệu; timing chặt	Pin peripheral bị giới hạn hơn; config phức tạp hơn

Nếu mục tiêu là W25Q, TFT hoặc streaming data tốc độ cao, hardware SPI thường tốt hơn. SoftSPI rất hữu ích để bring-up, test pin hoặc khi peripheral SPI phần cứng không còn available.

18. Tóm tắt để nhớ bản chất

Mental Model - Understand the Driver in 30 Seconds

- 1 **Init**
Turn GPIO pins into SCK/MOSI/MISO/CS and set idle levels.
- 2 **CS LOW**
Select exactly one SPI slave.
- 3 **For each bit**
Set MOSI -> toggle SCK -> sample MISO -> toggle SCK back.
- 4 **Repeat 8 times**
One transmitted byte and one received byte are completed.
- 5 **Buffer APIs**
Repeat the byte engine for N bytes.
- 6 **CS HIGH**
Close the transaction.

Software SPI = CPU manually creates the same four wires that a hardware SPI peripheral would generate automatically.

Figure 7. Mental model cuối cùng.

Một câu: SoftSPI_TransmitReceiveByte() chính là SPI engine: mỗi vòng lặp tạo hai cạnh SCK, xuất một bit MOSI, sample một bit MISO; lặp 8 lần thành một byte. Các API còn lại chỉ cấu hình GPIO, quản lý CS hoặc lặp byte engine cho buffer.